

UNIVERSIDAD EAFIT  
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA  
ARQUITECTURA DE SOFTWARE 2026

## Tutorial 02: Patrones Creacionales en Django

### *Factory Method y Builder para Sistemas Escalables*

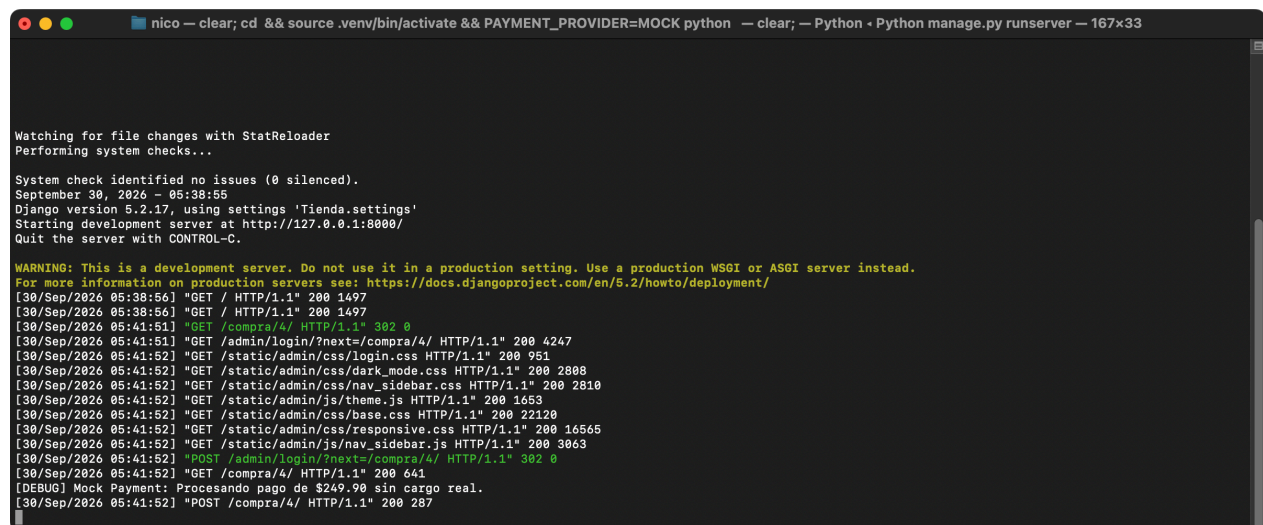
Nicolás Rodríguez

Profesor: Nicolás Ramírez Vélez

30 de septiembre de 2026

### 1. Evidencia: Mock Payment activado por variable de entorno

Levanté el servidor con `PAYMENT_PROVIDER=MOCK python manage.py runserver` y compré el libro #4 (Domain-Driven Design, \$210 + IVA) desde la vista `CompraView`. No toqué ni una línea de la vista: la `PaymentFactory` leyó el ambiente y entregó el `MockPaymentProcessor`. Resultado en consola:



```
nico — clear; cd && source .venv/bin/activate && PAYMENT_PROVIDER=MOCK python — clear; — Python • Python manage.py runserver — 167x33

Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
September 30, 2026 - 05:38:55
Django version 5.2.17, using settings 'Tienda.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/

[30/Sep/2026 05:38:56] "GET / HTTP/1.1" 200 1497
[30/Sep/2026 05:38:56] "GET / HTTP/1.1" 200 1497
[30/Sep/2026 05:41:51] "GET /compra/4/ HTTP/1.1" 302 0
[30/Sep/2026 05:41:51] "GET /admin/login/?next=/compra/4/ HTTP/1.1" 200 4247
[30/Sep/2026 05:41:52] "GET /static/admin/css/login.css HTTP/1.1" 200 951
[30/Sep/2026 05:41:52] "GET /static/admin/css/dark_mode.css HTTP/1.1" 200 2808
[30/Sep/2026 05:41:52] "GET /static/admin/css/nav_sidebar.css HTTP/1.1" 200 2810
[30/Sep/2026 05:41:52] "GET /static/admin/js/theme.js HTTP/1.1" 200 1653
[30/Sep/2026 05:41:52] "GET /static/admin/css/base.css HTTP/1.1" 200 22120
[30/Sep/2026 05:41:52] "GET /static/admin/css/responsive.css HTTP/1.1" 200 16565
[30/Sep/2026 05:41:52] "GET /static/admin/js/nav_sidebar.js HTTP/1.1" 200 3063
[30/Sep/2026 05:41:52] "POST /admin/login/?next=/compra/4/ HTTP/1.1" 302 0
[30/Sep/2026 05:41:52] "GET /compra/4/ HTTP/1.1" 200 641
[DEBUG] Mock Payment: Procesando pago de $249.90 sin cargo real.
[30/Sep/2026 05:41:52] "POST /compra/4/ HTTP/1.1" 200 287
```

Figura 1: Consola de Django con el mensaje [DEBUG] Mock Payment: Procesando pago de \$249.90 sin cargo real. justo antes del `POST /compra/4/` 200.

## Procesar Compra

¡Gracias por su compra! Total: \$249.90  
[Volver a la tienda](#)

Figura 2: Respuesta en el navegador: la compra se completó sin tocar el banco real ni escribir en el log de pagos.

## 2. Código fuente

### 2.1. infra/factories.py — Factory Method

Centraliza la decisión de qué procesador crear. La vista queda ciega respecto a la infraestructura (OCP: para agregar PayPal se extiende la fábrica, no la vista).

tienda\_app/infra/factories.py

```
import os
from .gateways import BancoNacionalProcesador

# Implementacion ligera para pruebas (Mocking)
class MockPaymentProcessor:
    def pagar(self, monto: float) -> bool:
        print(f"[DEBUG] Mock Payment: Procesando pago de ${monto} sin cargo real.")
        return True

class PaymentFactory:
    @staticmethod
    def get_processor():
        # LECCION ARQUITECTURA: La configuracion viene del ambiente, no del codigo
        provider = os.getenv('PAYMENT_PROVIDER', 'BANCO')

        if provider == 'MOCK':
            return MockPaymentProcessor()

        # Por defecto usamos la infraestructura real
        return BancoNacionalProcesador()
```

### 2.2. domain/builders.py — Builder

Ensambla la Orden paso a paso con interfaz fluida. Única adaptación frente al enunciado: el IVA se calcula con Decimal("1.19") porque el precio es DecimalField y multiplicar Decimal \* float en Python lanza TypeError.

tienda\_app/domain/builders.py

```
from decimal import Decimal

from ..models import Orden

class OrdenBuilder:
    def __init__(self):
        self.reset()

    def reset(self):
        self._usuario = None
        self._items = []
        self._direccion = ""

    def con_usuario(self, usuario):
        self._usuario = usuario
        return self # Permite Fluent Interface

    def con_productos(self, productos):
        self._items = productos
        return self
```

```

def para_envio(self, direccion):
    self._direccion = direccion
    return self

def build(self) -> Orden:
    if not self._usuario or not self._items:
        raise ValueError("Datos insuficientes para crear la orden.")

    # Encapsulamos la logica de calculo (Decimal: el dinero no se redondea "a ojo")
    subtotal = sum(p.precio for p in self._items)
    total_con_iva = (subtotal * Decimal("1.19")).quantize(Decimal("0.01"))

    orden = Orden.objects.create(
        usuario=self._usuario,
        libro=self._items[0], # el modelo registra un titulo por orden
        total=total_con_iva,
        direccion_envio=self._direccion,
    )
    self.reset()
    return orden

```

## 2.3. Integración en views.py y services.py (Pasos 2.2 y 3)

Extracto: views.py / services.py

```

class CompraView(LoginRequiredMixin, View):
    def setup_service(self):
        # ANTES: gateway = BancoNacionalProcesador()
        # AHORA: Delegacion total a la Fabrica
        gateway = PaymentFactory.get_processor()
        return CompraService(procesador_pago=gateway)

# services.py
def ejecutar_proceso_compra(self, usuario, lista_productos, direccion):
    orden = (self.builder
              .con_usuario(usuario)
              .con_productos(lista_productos)
              .para_envio(direccion)
              .build())

    if self.procesador.pagar(orden.total):
        return f"Orden {orden.id} procesada exitosamente."
    orden.delete()
    raise Exception("Error en la pasarela de pagos.")

```

Además dejé 7 pruebas unitarias (`python manage.py test`) que verifican que la fábrica devuelve el Mock con la variable activa y el banco por defecto, que el Builder calcula  $\$180 \rightarrow \$214.20$  y que rechaza órdenes sin usuario o sin productos. Todas pasan.

### 3. Reflexión

Crear la orden directamente en la vista funciona, igual que funciona calcular una nómina de mil empleados a mano: hasta el primer error. El **OrdenBuilder** existe porque prefiero que las reglas se cumplan solas a depender de que alguien las recuerde. La orden solo se crea cuando tiene usuario, productos y dirección, el IVA se calcula en **un único lugar**, y si falta algo, `build()` la frena antes de guardar o cobrar un peso. Nadie memoriza diez parámetros, nadie copia la fórmula del impuesto en cada vista y, si mañana cambia el IVA, se toca una línea. ¿Podía hacerlo todo en la vista? Claro. También podía escribir código que no escala. Prefiero no hacerlo.